

01-introduction

Background

- Automated systems are being used more and more. Hardware and software technology are improving very fast.
- But **user interface (UI) technology is not improving as fast**. This means today's biggest problem is not processing power—the **bottleneck is the user interface**.
- A human and a computer are both powerful information processors, but they communicate through a **narrow and limited interface** (keyboard, mouse, screen, etc.).
- **Goal:** Increase the “bandwidth” of communication between humans and computers.
- To do this, we should: First understand **human perceptual abilities** (how people see, hear, think, and react), Then design devices and interaction techniques. In short: **Design based on humans first, not machines**.

Computer Science and HCI: Problems and opportunities in Human–Computer Interaction (HCI) are growing as technology grows. Systems might be easier to use, but they are harder to program. There are still no simple engineering formulas to create good interfaces. This means: We must use iterative design, prototyping, and testing. The user interface often becomes the most frequently changed part of a software system.

What Is a Human–Computer Interface? It includes: All inputs from the user to the computer. All outputs from the computer to the user. The sequence/order of how inputs and outputs happen. Everything the user sees or perceives. It focuses on the user experience, not the internal implementation.

Usability means how well users can use the system's features. It includes: **Learnability:** Is it easy to learn? **Efficiency:** After learning, is it fast to use? **Visibility:** Can users easily see and understand the system's current state? **Errors:** Are errors rare and easy to recover from? **Satisfaction:** Is the system pleasant and enjoyable to use?

The Cost of Getting It Wrong Users' time is becoming more valuable. If you don't design the UI correctly now, you will pay later—either through complaints, fixes, or lost users. Good UI saves money.

Usability Is Only One Attribute of a System: Software designers must balance many things, such as: Functionality, Performance, Cost, Security, Usability, Size, Reliability, Standards. Many choices require **tradeoffs** between these attributes. In this course, an extreme focus is placed on **usability**.

Usability Dimensions Vary in Importance: What matters most depends on the **type of user**: **Novice users:** need learnability, **Infrequent users:** need memorability, **Expert users:** need efficiency
But no one is fully a novice or fully an expert. People may be experts in the domain but new to the application. Or they may know the application but not a specific feature.

The User Interface Is Important: The UI heavily influences how users **feel** about the software. A good UI helps the product sell better. A bad UI causes people to leave a website or uninstall software. Users often judge software based on surface-level impressions. Users often **blame themselves** for UI problems. Decision-makers who buy software are not always the ones who use it.

02-interaction-design

What is Interaction Design? The main goal of interaction design is to create **interactive products** that are **usable**. Usable means the product should be: **Easy to learn, Effective to use, Enjoyable for the user**

A good way to understand usable design is to: Compare well-designed and poorly designed products. Identify their strengths and weaknesses. By doing this, we can understand what makes something usable or not usable.

What to Design? Designing usable products requires understanding: **Who** will use them. **Where** they will use them. **What activities** the users will perform.

- The type of interface and input/output devices depends on the activity. Example: If people need to communicate over long distance, they need a system that: Allows easy input of messages (spoken or written). Allows users to access messages easily. Lets users edit, store, annotate messages
- People perform many types of activities using computers: Sending messages, Gathering information, Writing essays, Controlling machines, Drawing, Planning, Programming, Calculating, Playing games
- Similarly, there are many types of **interfaces and devices**: Multimedia applications, Virtual reality, Speech systems, Personal digital assistants, Large displays
- There are many **interaction styles**: Menus, Commands, Forms, Icons Etc.

- Designers face many choices when creating interactive products. The key question in interaction design is: **How do we optimize user interactions so they match and support the users' activities?**
- Designers can: Use intuition (guessing) and hope it works **OR** Use a more principled approach based on understanding users.

Better Design Through Understanding Users: To make the right design decisions, we must: Understand what people are good at and bad at. Think about how to support their existing ways of doing things. Think about how to create quality user experiences. Listen to users and understand what they want. Involve users by using tried and tested user-based design techniques.

The Process of Interaction Design: Interaction design has **four main activities**: Identify needs and establish requirements, Develop alternative design ideas, Build interactive versions (prototypes) to communicate and test ideas, Evaluate the designs throughout the process

- These activities inform each other and are repeated (iterative). Example: Evaluating usability may show that some requirements are not met → so changes must be made.
- Evaluation is at the heart of interaction design. The focus is making sure the product is usable. A user-centered design approach is usually adopted: Users are involved throughout the process.
- Many methods can be used to involve users: Observations, Interviews, Performance tasks, Modeling performance, Questionnaires, User testing, Making users co-designers
- Understanding users is important because: Different users have different needs Products must be designed accordingly
- In addition to the four activities, there are **three key characteristics**: Users must be involved throughout the project. Usability and user experience goals must be: Identified, Documented, Agreed upon at the start. Iteration (repeating activities) is unavoidable.

The Goals of Interaction Design: To design an interactive system, you must know your **primary objective**: Do you want high efficiency? Do you want motivation and engagement for learning? Or something else?

These objectives fall into: **Usability goals, User experience goals**

Usability Goals: Usability means ensuring interactive products are: Easy to learn, Effective to use, Enjoyable for users. Usability helps people perform activities in work, school, and daily life.

Usability goals include:

- **Effectiveness:** How well the system performs its intended functions.
- **Efficiency:** How quickly and smoothly tasks can be completed.
- **Safety:** Protects users from dangerous or undesirable conditions.
- **Utility (Usefulness):** Provides the right features so users can do what they need.
- **Learnability:** Easy for new users to learn the system.
- **Memorability:** Easy to remember how to use the system after not using it for a while.

User Experience (UX) Goals: These goals focus on the **quality of the user experience**, such as making it: Satisfying, Enjoyable, Fun, Entertaining, Helpful, Motivating, Aesthetically pleasing, Supportive of creativity, Rewarding, Emotionally fulfilling

04b-laws-of-hci.

1. **Fitts's Law** – Users hit targets faster when the target is big + close. Fitts's Law (created in 1954 by Paul Fitts) predicts how long it takes for a user to move and select a target. **The law says:** "Time to reach a target depends on the distance to it and its size." More distance → more time. Smaller target → more time. Even though the law was made before computer interfaces, it still works for GUIs today.
Design goal: Reduce the distance between interactive elements Make targets large enough for easy clicking. Make the entire element clickable (e.g., whole button area clickable, not just text). Example: Text links, images, or button areas should allow clicking anywhere inside the region.
2. **Power Law of Practice** – Users get faster with practice. **The power law of practice says:** Reaction time decreases as people practice more. The "logarithm of time" decreases in a straight line when plotted against the "logarithm of number of practice trials." This describes the learning curve. First proposed by Snoddy in 1928. Example: A graph where the x-axis is practice time. The y-axis shows reaction time dropping with more practice
3. **Pareto Principle (80/20 Rule): Pareto Principle says:** About 80% of results come from 20% of causes. Also called: The 80/20 rule, The law of the vital few, The principle of factor sparsity
In UX: "A small part of the site is used by most users." Meaning: focus on improving the critical 20% that most people use.

4. **Hick's Law** – More choices = longer decision time. **Hick-Hyman Law (by William Edmund Hick and Ray Hyman)** describes the decision time needed based on number of choices. More choices → Decision time increases logarithmically. Measures a user's cognitive information capacity.
Used often in UX/menu design, but: It only applies when choices can be divided in a predictable/logical way (like alphabetical menus). It does NOT apply when you need to scan every item one by one (like random lists). Example: If a menu is alphabetical and the user knows the name, they can find it faster (logarithmic time). But if random → they must check one by one (linear time).
5. **Miller's Law** – Short-term memory holds about 7 items (± 2). **From George A. Miller's famous 1956 paper:** People can store around 7 ± 2 items in short-term memory. **Two main limits discussed:**
 - A. **One-Dimensional Absolute Judgment:** A person is shown stimuli that vary on ONE dimension (e.g., 10 tones that differ only by pitch). People perform well up to 5 or 6 different stimuli; performance worsens when more are added.
The task works like an "information channel": Input: one of n stimuli. Output: one of n responses. Requires binary decisions → about 2–3 bits of information. Means people can reliably distinguish 4–8 alternatives
 - B. **Limits of Short-Term Memory:** Memory span = the longest list someone can repeat correctly 50% of the time. Young adults = usually about 7 items. Memory span is NOT based on number of bits: Binary digits = 1 bit each. Decimal digits = 3.32 bits. Words = ~10 bits. But memory span is still around 7 items.
This means short-term memory is about **chunks**, not bits. A **chunk** = the largest meaningful unit the person understands. Example: A word = 1 chunk for someone who knows the language. For a foreigner, it becomes many chunks.
About the "Magic Number 7": Miller said the "7" limit between judgment and memory was just a coincidence. He didn't mean the number 7 is truly magical. However, it became a useful heuristic: Long lists (much more than 7 items) become harder to remember. People cannot process too many things at once.

conceptual-models-2

Interaction design is made of three main parts:

1. **Concepts:** All ideas and objects that the user interface shows to users.
2. **Task Flow:** The step-by-step sequence of actions users do to complete tasks.
3. **Presentation:** The actual interface elements like buttons, controls, screens, and displays.

What Is a Conceptual Model? A conceptual model is a high-level description of the application.

It includes: All concepts a user will see in the application. How these concepts relate to each other. How these concepts fit into tasks the user performs. All data objects the user may encounter. The **operations, attributes, and relationships** the user can perform on these objects.

A conceptual model is independent of task flow and presentation. When you design task flow, you might find issues in the conceptual model and need to update it.

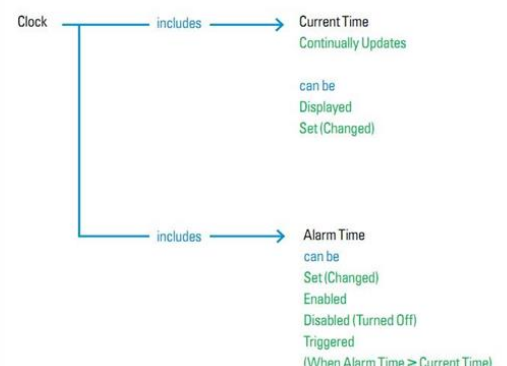
What Conceptual Models Should NOT Be: Lists are not models. Sentences are not good for showing structure because they can be repetitive and not visual enough.

Better Representation: Node-link diagrams (concept maps) are more efficient for showing conceptual models because they visually show structure, relationships, and connections.

Example: Johnson & Henderson Alarm Clock Conceptual Model

What the Alarm Clock Model Includes: The clock stores the current time. It continually updates the time as time passes. It always displays the current time. Users can set the current time. Users can set an alarm for a specific time or choose "no alarm". When the alarm is set and the current time equals the alarm time, the alarm triggers. Users can turn off the alarm.

A conceptual model of an alarm clock represented as a concept map.



Flexibility of This Model: This model works for: Digital travel clocks, Analog travel clocks, Digital clocks on phones like iPhone and Galaxy Nexus

Concept Map Versions: A version showing all major concepts (time, alarm, display, update, user actions). A more detailed version with elements connected more precisely.

Why Conceptual Models Matter (Summary): Design teams must agree on a conceptual model early, before wireframes or coding start. The conceptual model will change over time, which is normal. A standard format for conceptual models: makes them easier to understand, easier to make, easier to teach to new designers

- Conceptual model of an alarm clock with detail added.



interface-metaphors

What Is a Metaphor? A metaphor is when one thing represents another thing, usually something abstract. Metaphors connect things that are not directly the same. It is a figure of speech where a name or description from one thing is given to something else that is similar in some way.

Metaphors depend on: Difference (the two things are not identical) Analogy (there is some similarity that helps connect them) If there was no difference, it would not be a metaphor—it would be an identity. The difference shows they are not the same, and the analogy lets us think of one using the idea of the other. This helps us understand new concepts in useful and creative ways.

More About Metaphors: A metaphor explains a concept by comparing it to another concept. Important qualities: Metaphors are inexact (they are never perfect matches). They depend on context (culture, personal experience, environment). A metaphor works within a whole system of ideas, not alone. Metaphors may be one of the main ways humans understand new or complex concepts.

Metaphors and Computing: Computers are not the only technologies made easier through metaphors. Metaphors help people deal with the complexity of any new technology. When old technology is replaced by new, metaphors help us transition. New technologies bring new metaphors. Computers have many levels of understanding (hardware, OS, software, UI). People at each level—chip designers to everyday users—use metaphors to understand complexity.

The user interface (UI) uses the most metaphors because: Average users do not understand how computers work internally. Their understanding of the computer often depends completely on metaphors.

User-Interface Metaphors: A UI metaphor explains system functionality by connecting it to something the user already knows.

What is being explained? The hidden functioning and behavior of the software. **Definition:** A UI metaphor compares system functionality or structure to a familiar object or idea.

Example 1: The Trashcan Metaphor; “File deletion is like using a trashcan.” The trashcan helps users understand file deletion: Put unwanted items in it to get rid of them. Retrieve items if done quickly.

It uses visual and kinesthetic (movement) senses because users *drag* and *drop* items into it.

Example 2: The Desktop Metaphor; Used in modern graphical operating systems (Windows, macOS, Linux). Includes sub-metaphors: Trashcan, Documents, Desktop surface

The “desktop” metaphor is better described as an “office metaphor.” It helps users think of their computer like an office: Files, Folders, Documents

Metaphors can exist at different levels: Large system-level metaphors (desktop/office), Specific metaphors (trashcan), Subtle metaphors (traffic light metaphor)

Problems with Interface Metaphors: They can break cultural or traditional rules (Example: a recycle bin placed on the desktop). They can limit designers, making them stuck in existing ways of thinking. They may conflict with good design principles. Metaphors force users to understand the system only through that metaphor, even if it’s imperfect. Designers may copy bad designs from the real world into the digital world. Metaphors can restrict creativity, preventing designers from exploring better conceptual models.